Desarrollo de RentaMovil: Sistema de Gestión de Inventario y control de flota basado en arquitectura hexagonal y métricas de calidad

Nicole Dayana Ramirez Vargas
Servicio Nacional de Aprendizaje- SENA
Neiva, Colombia nicolerv18007@gmail.com

Thiago Fabian Rojas Guevara
Servicio Nacional de Aprendizaje- SENA
Neiva, Colombia thiagosrojasg451@gmail.com

David Santiago Erez Rodrgiuez Servicio Nacional de Aprendizaje- SENA
Neiva, Colombia erezsantiago25@gmail.com

1. Introducción

1.1 El Problema

En la pequeña y mediana empresa, el alquiler de vehículos suele resolverse con lo que haya a mano: una hoja de cálculo para la flota, una agenda de papel o un chat para las reservas, y la memoria para los pagos y el mantenimiento. El sistema aguanta un tiempo. Luego se rompe.

Eso fue, justamente, lo que encontramos: un desorden operativo. Nadie podía decir con certeza qué vehículos estaban libres en una fecha concreta. Las reservas se pisaban entre sí, porque no existía un punto único que validara la disponibilidad. Los pagos se daban por buenos sin un comprobante verificable, y el mantenimiento llegaba tarde, siempre reactivo, cuando el vehículo ya fallaba. A eso se sumaban dos huecos más: no había seguimiento de los vehículos durante el alquiler, y generar un documento formal —una factura, por ejemplo— resultaba imposible.

Hay una idea que la ingeniería de software repite hasta el cansancio y que aquí aplica de lleno: la calidad del producto depende, en buena medida, de la calidad de su diseño y de las prácticas con que se construye [Sommerville 2015]. No bastaba con que el sistema funcionara. Necesitábamos que su estructura se pudiera entender, mantener y verificar. Los problemas, en concreto, eran varios: información dispersa y sin centralizar; un ciclo de reserva imposible de rastrear de principio a fin; mantenimiento reactivo y sin planificación preventiva; conflictos de disponibilidad por no tener una única fuente de verdad sobre qué vehículo estaba ocupado; y control de acceso inexistente, sin roles ni permisos que separaran al cliente, al administrador y al superadministrador.

1.2 Motivación de la Investigación

La motivación de fondo no era solo resolver el alquiler. Queríamos que el sistema sirviera de caso de estudio: demostrar que un diseño disciplinado rinde software medible y mantenible. Por eso el equipo hizo lo contrario a "salir programando". Documentó el diseño antes de implementarlo (SDD), modeló el negocio con DDD y repartió responsabilidades con arquitectura hexagonal y microservicios [Evans 2003; Cockburn 2005; Newman 2021].

Hubo un segundo propósito. Desde el arranque definimos indicadores de calidad para evaluar, con cierta objetividad, el avance y la solidez del sistema. La literatura sobre DevOps apunta en esa dirección: los equipos que miden su desempeño de forma sistemática tienden a obtener mejores resultados [Forsgren et al. 2018; Forsgren 2021]. Nos pareció una afirmación comprobable y quisimos probarla en un proyecto académico real, con sus limitaciones y todo.

1.3. Objetivos de la Investigación

El objetivo general de esta investigación fue desarrollar un sistema de alquiler de vehículos que dejara atrás las limitantes del proceso manual, haciendo uso de las buenas prácticas de arquitectura y con la calidad del resultado medida, no con valores supuestos.

De ahí se desprendieron seis objetivos específicos:

- Modelar el dominio del negocio y reconocer contextos acotados con DDD.
- Implementar microservicios con un API Gateway y arquitectura

hexagonal interna.

- Establecer un modelo de datos normalizado, con el esquema bajo control de versiones.
- Asegurar el sistema con tokens JWT, roles y permisos.
- Construir los clientes web y móvil sobre contratos de API documentados.
- Definir y evaluar un conjunto de indicadores de calidad.

1.4. Importancia del Tema

La gestión eficiente de la flota y las reservas no es un lujo: es competitividad. La automatización de estos procesos empuja a que reserva, pago y entrega sean confiables y rentables. En el caso colombiano, adoptar buenas prácticas de ingeniería y de gestión tecnológicas pesa directamente en la productividad de las empresas [Espinosa and Eraso 2024]. Rentamovil pone esos principios a prueba en el sector de la movilidad, y su documentación queda como referencia para cualquier empresa que quiera intentar algo parecido.

2. Trabajos Relacionados

Esta sección repasa los seis frentes que sostienen el diseño de RentaMovil: microservicios, arquitectura hexagonal, diseño guiado por el dominio, métricas de calidad, patrones de diseño y seguridad web.

2.1. Arquitectura de Microservicios

Para Newman [Newman 2021], los microservicios son un estilo que parte la aplicación en servicios pequeños, desplegables por separado y organizados alrededor de funcionalidades del negocio. [Fowler and Lewis 2014] añaden el otro lado de la balanza: el estilo gana cohesión funcional y autonomía de equipos, pero cobra el precio en operación y en la comunicación entre servicios, que hay que gestionar. Ahí entran los patrones de Richardson [Richardson 2018] -API Gateway, base de datos por servicio, consumidores idempotentes, publicación de eventos, pensados justamente para bajar ese costo. RentaMovil recoge varios: puntos de entrada únicos, esquema de base de datos por servicio y consumidores de eventos idempotentes.

Piñero González et al. [Piñero González et al. 2021] identifican que la eficiencia del desempeño en productos de software está relacionada con

2.2. Arquitectura Hexagonal (Puertos y Adaptadores)

Cockburn [Cockburn 2005] ideó la arquitectura hexagonal para algo sencillo y difícil a la vez: aislar el negocio de la tecnología. El dominio declara puertos (interfaces); la infraestructura aporta adaptadores que los enchufan a bases de datos, servicios web o mensajería. Martin [Martin 2017] lo lleva a una regla: las dependencias apuntan hacia el dominio, nunca al revés, y eso es la inversión de dependencias. En RentaMovil cada microservicio respeta la separación —los paquetes domain no conocen Spring ni JPA— y no por dogma, sino porque nos ahorra dolor cuando algo externo cambia.

2.3. Diseño Guiado por Dominio (DDD)

Evans [Evans 2003] dejó tres conceptos que usamos casi a diario: lenguaje ubicuo, contexto acotado y mapa de contexto. La idea es que el software hable el idioma del negocio. En RentaMovil los límites de cada servicio salen de sus contextos acotados (Identidad, Flota, Reservas, Renta, Pagos, Telemetría y Notificación). El desacoplamiento, así, descansa en el dominio y no en una moda tecnológica de turno

2.4 Métricas de Calidad y DevOps

Forsgren, Humble y Kim [Forsgren et al. 2018] no se quedaron en la teoría: mostraron con datos que la entrega continua y una arquitectura desacoplada anticipan mejor el rendimiento. Sus métricas DORA -frecuencia de despliegue, tiempo de entrega, tasa de fallos y tiempo de recuperación- terminaron siendo un estándar en la industria [Forsgren 2021]. Humble y Farley insisten en dos cosas

las cuales asumimos, pero en realidad no podemos dejarlas de lado: automatización y retroalimentación rápida. Tomamos esas ideas y las adaptamos en un panel KPIs de ingeniería a la medida de un proyecto académico.

2.5 Patrones de Diseño y Separación de capas

Los patrones clásicos [Gamma et al. 1994] y los de arquitectura empresarial [Fowler 2002] Son soluciones ya explicitas para problemas comunes y repititivos: Repository utilizado para el acceso de datos, Data Tranfer Object para transferir información entre capas, Service Layer para aislar la lógica de negocio. Parnas [Parnas 1972] lo habia dicho mucho antes: la buena división – la que oculta las decisiones de diseño que van a cambiar- es la que sostiene la mantenibilidad. RentaMovil los aplica sin inventar nada.

2.6 Seguridad en Aplicaciones web

Un sistema que maneja datos personales y transferencia de pago no puede tomarse con tal libertad y ligereza la seguridad de estos datos sensibles. OWASP [OWASP 2021] la cual enumera las 10 vulnerabilidades de seguridad más críticas: autentificación rota, exposición de datos sensibles, fallas de control de acceso, inyecciones etc. El cómo RentaMovil responde antes estas vulnerabilidades, combina JTW firmados, hash de contraseñas con BCrypt, autorización por permisos, validación de entrada y consultas parametrizadas vía JPA.

3. Metodología

3.1 Enfoque Metodológico

Para esto convergieron tres prácticas. La primera **SDD (Software Design Documentation)**: Antes de empezar a estructurar el código el diseño se debe documentar y se mantiene vivo con cada modificación, evoluciona con el sistema no se archiva. L a segunda, DDD: el modelo del negocio se levantó antes del código, con un mapa de contextos, entidades, invariantes y eventos de dominio. La tercera, Scrum: como metodología considerada marco de gestión ágil, basada en historias de usuario las cuales se emplearon con el formato Give/When/Then, puntos de historia, criterios de aceptación y una lista de verificación compartida por equipo (DoD). Además de un hábito general: cada decisión arquitectónica relevante quedo registrada como una ADR (Architecture Decision Record) de tal forma que se tiene registro de que alternativas se evaluaron, cuales se eligieron y cuales se descartaron.

3.2 Arquitectura del sistema

RentaMovil funciona gracias a una comunicación cliente-servidor y un API Gateway como único punto de entrada y comunicación entre servicios pro medio de contratos REST. Los clientes, la web de cliente, administrativa y la app móvil, no hablan con los servicios directamente: pasan por el Gateway, que enruta cada petición por la ruta hacia el microservicio que corresponde, valida el token JWT y aplica límites de tasa. Cada microservicio implementa su propia arquitectura hexagonal los cuales desacoplan su lógica de negocio de los detalles de su infraestructura. La base de datos relacional es una sola a nivel físico, pero cada microservicio posee su propio esquema claramente con sus respectivas reglas de acceso.

La figura 1 resume la arquitectura referenciada.

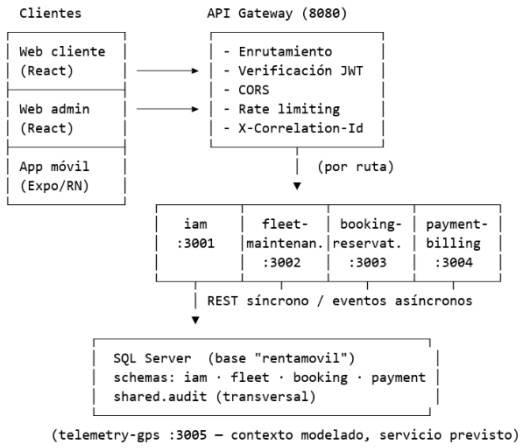


Figura 1. Arquitectura de referencia de RentaMovil: microservicios, Api Gateway y esquema por servicio.

3.3 Dominios Y Descomposición en Servicios

El dominio se parte en siete contextos. Ahora bien; un contexto no tiene que equivales a un servicio desplegable, por economía de despliegue, además de que no todos los contextos justifican su propio proceso de despliegue. Por ello agrupamos aquellos contextos que comparten ciclo de vida. Reservas, ejecución de la renta y notificación, por ejemplo, viven juntos. Una reserva y su renta son la misma transacción con dos estados. Separarlas solo habría añadido llamadas a la red sin ganancias.

La tabla 1 ordena la relación entre contextos, servicios y tablas.

#	Contexto/Servicio	Responsabilidad	Puerto	Tablas propias
01	Api Gateway	Punto único de entrada: enrutamiento, autentificación, limites.	8080	-
02	Iam	Registro, login, tokens JWT, roles y permisos	3001	Person, user, role, permission, role_permission, verification_code,sesión
03	Fleet-maintenance	Catálogo de vehículos, sucursales, mantenimiento	3002	Branch, Branch_operating_hour, Brand, category, engine_type, vehicle_model, vehicle, maintenance_type, vehicle_maintenance
04	Booking-reservation	Reserva, ejecución de la renta, notificaciones y seguros	3003	Insurance_type, reservation, rental, notification
05	Payment-billing	Pagos por transferencia, revisión de comprobantes, facturas	3004	Bank_account, payment, invonce
06	Telemetry-gps	Captura de ubicación y historial de ruta.	3005	Gps, location (previsto)

Tabla 1. Contextos acotados, servicios desplegables y tablas propias.

3.4 Selección de Tecnologías

Las tecnologías se eligieron por madurez, rendimiento y por supuesto por la increíble facilidad de mantenimiento. El backend quedo en java 21 con Spring Boot; una infraestructura probada, con dependencias, además de seguridad y acceso a datos con valores de fábrica. ¿Es necesario unificar todas las versiones de todos los servicios? Por supuesto que no, cada uno es independiente y fija sus versiones, tal como ocurre con el Gateway. Para almacenar datos de forma normalizada en Microsoft SQL Server, empelando Liquidbase encargándose del versionamiento de la base de datos. La calidad de la base se sostiene con claves foráneas, restricciones CHECK y usuarios de aplicación con permisos dados en el propio esquema.

En el Frontend, se empleó React con vite para la web (modelo de componentes y un ecosistema bastante amplio) y Expo + React Native, el cual permite compartir estilos entre iOS, Android y web. Para el despliegue local se usó Docker y Docker compose, basado en una red donde los servicios se encuentran por nombre.

3.5 Patrones de Diseño Aplicados

3.5.1 Puertos y Adaptadores

Cada servicio separa tres zonas: domain (entidades, invariantes, puertos), appplication/usecase (los casos de uso) e infraestructura (adaptadores de persistencia, de seguridad, web y clientes). El domino no conoce a Spring ni JPA. Para poder cambiar la base de datos, de bróker o de cliente REST no se modifica nada más que un adaptador; las reglas de negocio ni se enteran de este cambio.

3.5.2 Repository y Service Layer

El acceso a datos se logra por medio de los repositorios (interfaces Spring Data JPA) detrás de puertos de salida. Mientras tanto, la lógica vive en los servicios y casos de uso; los controladores no tienen reglas. El poder probar con dobles (mocks, fakes) se vuelven un beneficio ya que estas pruebas son sencillas y el código se reutiliza sin arrastrar o afectar dependencias.

3.5.3 Data Transfer Object (DTO)

Los controladores hablan en DTO de entrada y salida. Ni las entidades de dominio ni las de la de JPA asoman por la API. Así podemos controlamos que se expone, validamos lo que entra además de poder rehacer el modelo interno sin necesidad de romper el contrato de cara al cliente.

3.5.4 Publicador de eventos de dominio y consumidor idempotente

Los servicios publican eventos de dominio como: ReservationCreated y RentalCompleted y esos eventos los consumen otros endpoints internos. La entrega, ya se sabe, puede repetirse. Por eso los consumidores son idempotentes; el tener que procesar dos veces el mismo evento no modifica el sistema, ya que lo deja igual que tener que procesarlo una sola vez.

3.5.5 API Gateway y esquema por servicio

El Gateway unifica el enrutamiento y la seguridad del borde. Quien define quien es el dueño de cada tabla es el esquema de cada servicio el cual, también permite aislar los permisos de base de datos por servicio (lo documentamos en el ADR-009).

3.6 Seguridad Implementada

La seguridad se pensó por capas. La autentificación se delega en tokens JTW firmados con RS256, el servicio iam es quien se encarga de guardar la clave privada y pública la publica en GET / jwks; los demás servicios verifican la firma con esa clave, de tal forma que nadie más que iam puede emitir tokens. El acces token dura 30 minutos solamente y el refresh token, 7 días, con rotación de un solo uso. Asi, si llega un refresh ya consumido, se tumban todas las sesiones del usuario. La autorización es de grano fino con permisos en formato entidad:acción, por ejemplo, roles:grant, y tres roles fijos, los cuales son ADMIN, CLIENT y SUPER-ADMIN. Si se hacen cinco intentos fallidos de inicio de sesión la cuenta es bloqueada; solo un superadministrador la desbloquea.

3.7 Sistema de métricas (KPIs)

Para poder asegurarnos de medir de manera adecuada la calidad de ingeniería de nuestro software, definimos diez indicadores con metas de la industria claramente ajustadas a lo que un proyecto formativo puede realmente sostener. Para evitar cuestionamientos subjetivos, como el software es lento o poco confiables, los indicadores cubren cuatro dimensiones: *rendimiento* (latencia de endpoints críticos), *confiabilidad* (disponibilidad objetivo y recuperación ante fallos), *calidad del código* (cobertura de pruebas, acoplamiento del dominio, seguridad) y *proceso* (trazabilidad, documentación, idempotencia).

3.8 Buenas prácticas aplicadas

Para poder cumplir con las ides de código limpio [Martin 2008] se aplicaron funciones cortas con una sola responsabilidad, comentarios cuando solo aportan contexto. Fuimos rigurosos y respetamos los principios SOLID, sobre todo aplicando la inversión de dependencias y responsabilidad única. Para proteger la integridad de los datos se protegen varios frentes a la vez. La validación en el formulario del cliente, validación del DTO con Bean Validation en el controlador, reglas de negocio en el caso de uso y restricciones de base de datos como ultima barrera. Para el versionamiento y colaboraciones del código, convenciones de ramas (feat/, Fix/) revisiones por Pull Request con Request con Squash and Merge y una definición de terminado compartida.

4 implementación

4.1 Estructura Común del Backend

Todos los servicios comparten el mismo esqueleto hexagonal: domain/ para las entidades, enumeraciones, excepciones y puertos de entrada y salida; appplication/usecase/ con cada caso de uso e infrastructure/ con configuración, persistencia, seguridad, clientes externos y la capa web. Una regla que lo refuerza es que el servicio nunca toca el esquema (spring.jpa.hibernate.ddl-auto=validate), solo comprueba que las entidades coordinen campos con las tablas que crearon las migraciones. Cualquier cambio de esquema empieza con un changeset en el repositorio de base de datos que corresponda.

4.2 Servicio IAM (identidad y Acceso)

El servicio iam permite el registro de clientes, el inicio de sesión, la renovación de tokens, la recuperación de contraseña, el perfil y la administración de roles y permisos. Expone POST /register, POST /login, POST /refresh, POST /password/forgot, PATCH /users/me y las operaciones de administración PATCH /users/{id}/role y PATCH /users/{id}/status. Al arrancar el servicio, se validan las llaves JWT: si alguna no es válida o tiene menos de 2048 bits, el sistema no inicia. Por seguridad, el primer superadministrador se da de alta a mano mediante un proceso independiente. Así evitamos dejar contraseñas o hashes expuestos en el repositorio y garantizamos que las credenciales de sesión no se identifiquen en texto plano.

4.3 Servicio Fleet & Maintenance (Flota de Mantenimiento)

Gracias a este servicio se determina que vehículos existen, al igual que se controla su estado y donde están. Se administran los vehículos y sucursales, manejando el horario de operación de estas. Se administran así también, las marcas, modelos, categorías y tipos de motor, además del ciclo de mantenimiento. Los vehículos tienen cuatro estados, AVAILABLE, RENTED, MAINTENANCE y RETIRED. Su disponibilidad no se guarda se consulta. Dos reglas de negocios: un vehículo ocupado no pasa a mantenimiento, y RETIRED es terminal, porque las filas nunca se borran físicamente.

4.4 Servicio Booking & Reservation (Reservas y Rentas)

Se define el ciclo de vida de cada reserva, se crea, consulta, modifica (solo la sucursal de devolución) se cancela y se deja expirar sola; además del registro de entrega y devolución del vehículo y de las notificaciones internas. Los estados de cada reserva son: PENDING_PAYMENT → PENDING_REVIEW → CONFIRMED → COMPLETED, con salidas hacia CANCELLED. Los totales son la suma del subtotal del vehículo y del seguro. Las reglas de negocio se concentran únicamente en el dominio. Por ejemplo, la sucursal de devolución solo se puede modificar con tres días de anticipación al finalizar la reserva. Una reserva sin comprobante de pago se cancela cuando se vence el plazo de 24 horas. **

4.5 Servicio Payment & Billing (Pagos y Facturación)

El sistema de pago aplicado en RentaMovil no es una pasarela de pago, debido a como está orientado el proyecto. En su lugar se implementó el pago por transferencia. El cliente transfiere el valor exacto de la reserva a una de las cuentas de la empresa y sube el comprobante; el administrador lo revisa y lo aprueba o lo rechaza. El servicio se encarga de comprobar que el monto coincida con el valor total de la reserva, no puede haber pagos parciales. Con este servicio, el rol SUPERADMIN puede manejar todas las cuentas bancarias, asimismo, emite la factura con numeración secuencial. Y para mantener la coherencia con el esquema por servicio, el monto se solicita a través de la API de booking-reservation; nunca se realiza un JOIN directo con su tabla.

4.6 Servicio Telemetry & GPS (Telemetría)

Rastrea el vehículo mientras esta rentado y conserva el historial de rutas. Es el que más escribe, un dispositivo reporta posiciones sin parar durante el alquiler. Debido a esto, es el candidato perfecto a una base de datos de series temporales si el volumen algún día lo justifica.

4.7 API Gateway

El Gateway es el único expuesto fuera de la red interna. Enruta por ruta, /auth/ hacia iam, además de verificar la firma y la vigencia del token, inyecta los identificadores de usuario y rol hacia los servicios, aplica CORS y añade un X-Correlation-Id a cada petición para poder rastrearla en los registros y aplica límites de tasa. Si un servicio no responde, devuelve 503; si se pasa del tiempo de espera, 504. Para mayor seguridad, se agregó un detalle de diseño, cada servicio vuelve a verificar el token antes de autorizar cualquier operación.

4.8 Persistencia y Estrategia de Migraciones

Para evitar base de datos individuales, la base de rentamovil es una sola a nivel físico, pero ordenada en esquemas por servicios: iam, fleet, booking y payment, mas shared para la auditoría. Para lograr esto cada servicio tiene su propio repositorio de migraciones (rtm-<servicio>-db) con Liquidbase y su historial de cambio aparte. En cada repositorio se fijoo un orden por tipo de sentencia: 01_ddl (esquemas, tablas sin claves foráneas, luego claves foráneas e índices), 02_dml, 03_dcl (usuarios y permisos de base de datos), 04_tcl y 05_rollbacks. Entre repositorios, el orden es iam → fleet-maintenance → telemetry-gps → booking-reservation → payment-billing, con esto las claves foráneas entre servicios apunten siempre a tablas existentes. El servicio se conecta con un usuario restringido a su propio esquema para que así la propiedad de datos y la base compartida dejan de contradecirse.

4.9 Frontend Web

La web se construyó React y Vite, empelando la arquitectura feature-first con su organización por funcionalidades y arquitectura de componentes. Cada funcionalidad (autenticación, catálogo, reservas, pagos, administración, reportes) reúne componentes, hooks, servicios, utils y paginas; lo reutilizable vive en un directorio compartido. Para evitar

el retraso por parte del Frontend mientras el backend se construye, construimos un servidor de datos simulados. La interfaz incluye internalización, formularios validados, mapas para la ubicación y gráficos para los paneles administrativos.

4.10 Aplicación móvil

Para construir la aplicación móvil se usó Expo y React Native, con Expo Router para la navegación y Zustand para el estado global. Está tipado con TypeScript y se organiza por funcionalidades, al igual que el Frontend. La navegación no conoce la lógica de negocio y solo los servicios pueden encapsular las llamadas a la API.

4.11 Flujo de Petición

Vamos con el recorrido de una petición. El usuario pulsa un elemento del Frontend y el cliente envía un HTTP con el token de acceso. El Gateway valida el token y ruta, luego reenvía al servicio asignado con un identificador de relación. El controlador del servicio repite la verificación del token, valida el DTO de entrada y delega en el caso de uso. Las reglas de negocio se aplican y se usan los puertos de salida para leer o escribir. La petición escala y se vuelve un JSON consistente. Los errores también van uniformes: código, mensaje y detalle de validación.

4.12 Comunicación entre servicios y Eventos de Dominio

La comunicación es sincrónica cuando hace falta consistencia inmediata, como consultar la disponibilidad de un vehículo antes de crear la reserva. Se aplica la comunicación asincrónica cuando la repuesta del sistema puede esperar, como generar una notificación al confirmarse un pago. Cada evento de cada dominio se publica con un sobre común y se consumen en los endpoints internos que no salen de la red. La tabla 2 resume los eventos principales y los dispara.

Evento	Publicado por	Consumido por / efecto
ReservationCreated	booking-reservation	Notificación al administrador
PaymentCompeted	payment-billing	Reserva pasa a CONFIRMED, notificación al cliente
PaymentRejecte	payment-billing	Reserva vuelve a PEDING_PAYMENT o se cancela, según el resultado
InvoiceGenerated	payment-billing	Notificación con el enlance de descarga de la factura
RentalStarted	booking-reservation	Fleet-maintenance marcar el vehículo como RENTED; telemetría inicia el rastreo
RentalCompleted	booking-reservation	Fleet-maintenance marca el vehículo como AVAILABLE; telemetría detiene el rastreo
MaintenanceCompleted	fleet-maintenance	booking-reservation crea la notificación correspondiente

5. Pruebas y Validación

5.1 Estrategia de Pruebas

La estrategia tiene forma de pirámide, se realizan pruebas unitarias en la base, pruebas de contrato e integración en el medio, además de unas pocas de extremo a extremo arriba [Cohn 2009; Fowler 2012]. La proporción meta es realizar 50% unitarias, 20% de contrato, 25% de integración y 5% de extremo a extremo. Esa distribución mantiene la integración estable y, asimismo, evita el antipatrón de darla la vuelta a la pirámide. No damos porcentajes de cobertura porque JaCoCo [JaCoCo s. f.] aún no está configurado, así que informamos cuantas pruebas se ejecutaron y cuántas pasaron. Los repositorios son proyectos Maven independientes, sin un POM agregador. Cada módulo se prueba por separado con JDK 21 y mvn test.

5.3 Pruebas del Backend

Cada prueba unitaria se realiza con Junit 5 t Mockito y AssertJ [Meszaros 2007]. Se aíslan los casos de uso de Spring y reemplazan las dependencias externas por dobles de prueba. Lo de mayor importancia son las reglas de negocio, además de los errores y las interacciones relevantes. La tabla 3 muestra la línea base ejecutada el 9 de octubre de 2026.

Módulo	Comportamiento ejercitado	Pruebas	Fallos/Errores
iam	Registro, login, logout, refresh de token, perfil, correo, contraseña, roles y estado	34	0 / 0
fleet-maintenance	Vehículos, disponibilidad, mantenimiento, sucursales, eventos de renta, verificación JWT y consultas JPA con H2	114	0 / 0
booking-reservation	Reservas, rentas y seguros, eventos de pago, expiración de reservas	53	0 / 0
payment-billing	Entidad Payment y caso de uso de pagos	14	0 / 0
api-gateway	Filtro JWT, correlation ID y limitador de tasa	10	0 / 0
Total	33 clases de prueba	225	0 / 0

Tabla 3. Resultados de las pruebas automatizadas del backend.

Las 225 suman cinco ejecuciones de Maven, no una sola ejecución multimódulo. Billing es el módulo con menos pruebas.

5.3 Pruebas de Persistencia, Controladores y Contrato

Fleet tiene un con conjunto de pruebas con @DataJpaTest sobre H2. Se cubre las consultas de sucursales, vehículos, mantenimiento y modelos. H2 ayuda a validar el comportamiento de JPA y JPQL, pero no reemplaza a SQL Server. Se deja fuera el esquema shared.audit, el cual no existe en H2 y hace que Hibernate registre un aviso. IAM, Booking y Billing usan JPA en producción.

Algunas pruebas tratan los filtros como unidades aisladas y otras llaman a los controladores directamente. Esto sirve para probar la delegación de y la lógica local. No pasan por el servidor MVC, así que no se logra verificar rutas HTTP, la cual no incluye serialización JSON ni valida la entrada. En

los cinco módulos no se realizan pruebas con @WebMvcTest o MockMvc.

Cada servicio contrato tiene un contrato OpenAPI [OpenAPI Initiative s. f.] lo que sirve como documentación y como acuerdo entre productor y consumidor. Aun no existen pruebas automáticas que comprueben que la implementación lo cumple.

5.4 Pruebas del Frontend

Como línea base automatizada solo cubre el backend. Mientras se desarrollaba el Frontend, para no evitar modificación de estructura, se creó un servidor de datos simulados, que permitía reproducir errores y estados vacíos sin dependencia a los servicios. No hay conjunto de pruebas automatizadas para la web o móvil.

5.5 Pruebas de Integración y Extremo a Extremo

La evidencia de la integración se validó a mano, así que se mantiene separada de la automatizada. El 4 de octubre se hizo una verificación con Docker, SQL Servevr, Gateway y Postman, con 37 de 37 solicitudes exitosas. Con el sistema ya desplegado, el endpoint de salud del Gateway, el 9 de octubre respondió ok con los cuatro servicios. El Frontend web cargó con HTTP 200 Y el equipo recorrió el flujo de la app móvil instalada contra del backend público.

Gracias a esa integración manual se hicieron diez hallazgos correctivos (C-01 a C-10) los cuales las pruebas automatizadas no habían detectado. Varios errores se causaban entre lo que enviaba el cliente y lo que esperaba el backend, como nombres de campos distintos, rutas mal supuestas y un refresh token rotado que el cliente no guardaba, lo que causaba un bucle de inicio de sesión. Esas fallas las habría detectado las pruebas de contrato y las de MockMvc. Los flujos críticos que se recorrieron fueron la creación de una reserva y la revisión de comprobante de pago, ya que no está automatizado.

5.6 Pruebas de Seguridad

No se realizó una auditoría OWASP completa. Algunos controles tienen respaldo en pruebas automatizadas, y otros existen solo como decisiones de diseño. Tabla 4 los separa.

Control	Evidencia
Verificación de JWT en el borde y en los servicios	Automatizada: JwtAuthenticationFilterTest (gateway) y JwksTokenVerifierAdapterTest (Fleet)
Límite de solicitudes por IP	Automatizada: InMemoryRateLimiterTest
Identificador de correlación	Automatizada: CorrelationIdFilterTest
Login, logout, refresh, contraseña, roles y estado de usuarios	Automatizada: casos de uso de IAM
CORS restringido, BCrypt, consultas parametrizadas, secretos en variables de entorno, permisos mínimos por esquema	Decisión de diseño, sin prueba automatizada

Tabla 4. Controles de seguridad y su evidencia.

5.7 Trazabilidad

El proyecto rentamovil mantiene una matriz que conecta cada requisito funcional con su historia de usuario y su servicio. Se tienen 31 historias en siete épicas, cada una con sus criterios de aceptación verificables. La columna de pruebas de esa matriz se llenó antes de implementar y todavía no está con las pruebas reales. La matriz marca 32 filas como pendientes y una en progreso. Ponerla al día con las 33 clases que existen es una acción de mejora abierta.

6. Resultados

6.1 Producto Construido

Se construyó un sistema distribuido en cuatro servicios de negocio y un API Gateway con dos clientes (web, móvil) y una base relacional compartida con un esquema por servicio. La tabla de 5 reúne las métricas técnicas que salen de los repositorios.

Microservicios de negocio + gateway	4 + 1 (telemetría modelada, no implementada)
Contextos acotados modelados	7
Tablas de dominio (schemas iam, fleet, booking, payment)	23 (+ auditoría transversal)
Archivos Java de producción (backend)	399
Clases de prueba / pruebas automatizadas	33 / 225
Endpoints REST documentados	Más de 60
Historias de usuario	31 en 7 épicas
Contratos OpenAPI	7 (uno por servicio, incluido telemetría, más un contrato compartido)
Registros de decisión arquitectónica (ADR)	9
Archivos del cliente web / móvil	305 / 146
Acciones correctivas, preventivas y de mejora registradas	10 / 5 / 5
Microservicios de negocio + gateway	4 + 1 (telemetría modelada, no implementada)

Tabla 5. Métricas técnicas del proyecto.

6.2 Indicadores de calidad

Se utilizaron diez indicadores parar realizar la evaluación con la evidencia que había al cierre de la fase analizada. Donde no había evidencia, el indicador se dejó sin medir.

#	Indicador	Meta	Estado	Evidencia
1	Cobertura de pruebas	≥ 80 %	○ Sin medir	JaCoCo no configurado; 225 pruebas pasan
2	Latencia p95 en endpoints críticos	< 2 s	○ Sin medir	Validación manual; sin pruebas de carga (NFR-001)
3	Disponibilidad	≥ 99 % mensual	○ Pendiente	Sin producción real (NFR-002)
4	Seguridad frente a OWASP Top 10	0 hallazgos críticos	🟡 Parcial	Controles implementados y probados en parte; sin auditoría
5	Portabilidad (contenedores)	100 % de servicios	🟢 Cumplido	Dockerfile en los cinco módulos
6	Acoplamiento del dominio	0 dependencias de framework	🟢 Cumplido	Paquete domain sin Spring ni JPA
7	Trazabilidad requisito → historia → prueba	100 %	🟡 Parcial	Requisito → historia → servicio completo; columna de pruebas por actualizar
8	Idempotencia de consumidores de eventos	100 % de eventos	🟢 Cumplido	Consumidores idempotentes por estado

9	Documentación por servicio	100 % con README y contrato	🟢 Cumplido	Siete contratos OpenAPI y rtm-docs
10	Cobertura funcional del MVP	≥ 80 %	🟡 En progreso	Cuatro de cinco servicios operativos

Tabla 6. Panel de indicadores de calidad del sistema.

Cuatro indicadores cumplidos, tres parciales y tres sin medir. Si cada uno vale 1, 0,5 o 0 puntos, el índice de madurez de ingeniería da 55 sobre 100. Ese número nos dice cuántas de las practicas que el equipo se propuso se cumplieron. No indica nada sobre el rendimiento en producción.

6.3 Cumplimiento Funcional

El 9 de octubre de 2026 se desplegó la plataforma con Docker, niginx y ngrok. Todos los servicios reales respondieron a través del Gateway, el Frontend web cargó y la app móvil e instaló como APK, asimismo se probó contra el backend público. Identidad, flota, reservas y pagos funcionan entre sí y los clientes consumen sus contratos. El contexto de telemetría sigue sin implementarse; por esa razón Frontend no refleja los datos enviados del GPS. El flujo principal comprende el consultar catálogo, verificar disponibilidad, crear reserva, subir comprobante, aprobarlo, confirmar la reserva y registrar la entrega y devolución. El cual las historias de usuario lo cubren al igual que las reglas de dominio probados Booking.

7 Discusión

7.1 Impacto de la Arquitectura Hexagonal y de los Microservicios

Al implementar la arquitectura hexagonal en cada microservicio, se pudo hacer notable la mantenibilidad del código. Al separa el que hace, en el dominio; el cómo se hace en la aplicación y el con que se comunica en la infraestructura, se deja claro donde se debe hacer cada cambio. Una regla de negocio vive en el dominio, y un cambio se tecnología se vuelve en un adaptador. Así también, la frontera entre servicios la marcó el dominio y no las pantallas, para así lograr evitar que la interfaz termine dictando la arquitectura. Lo compartido se quedó con la API Gateway (autentificación, perimetral, CORS, límites y correlación), así cada servicio puede ocuparse de lo suyo.

7.2 Análisis de los KPIs

Los indicadores que mejor muestran el valor del diseño son el acoplamiento del dominio, la portabilidad y la documentación, ya que se verifican de forma estática. Un dominio aislado se prueba sin alterar a lo demás, y un servicio contenerizado se despliega sin mayores inconvenientes. Aun no se aplican las herramientas de medición para la cobertura, latencia y disponibilidad. La trazabilidad quedó incompleta ya que incluso con la matriz existente las pruebas no se actualizaron.

7.3 Decisiones Arquitectónicas y Compromisos

Las decisiones importantes quedaron como ADR, junto con las alternativas que se descartaron. La primera decisión fue usar una base de datos compartida por un esquema por servicio, en lugar de complicarnos con una base de datos por servicio. De esta manera se gana integridad referencial entre contextos y se evitan las transacciones distribuidas, a cambio de coordinar el orden las migraciones. La segunda fue que al momento de consultar la disponibilidad de un vehículo de forma sincrónica en lugar de mantener una copia local actualizada de eventos. Se escogió un pequeño acoplamiento a incluir una inconsistencia como es que el cliente pueda consultar un vehículo en estado de reserva. La tercera fue estructurar los contextos con el mismo ciclo de vida en un solo servicio. Evitamos la sobrecarga de coordinación de red que le quita escalabilidad al proyecto. Una arquitectura eficiente nade de compromisos explícitos y fundamentados, no de la aplicación sistemática de patrones sin sentido o coherencia.

7.4 Valor de la Documentación como Parte del Diseño

Fue de gran importancia utilizar el enfoque SDD, el documentar antes de implementar, permitió discutir los límites y el alcance entre servicios y los contratos de API cuando cambiarlos todavía costaba poco. Al momento de definir las reglas de negocio como reglas fijas y las historias con criterios verificables redujo las inconsistencias y contradicciones al momento de programar. El tener

como fuente de verdad la documentación mientras se desarrollaba el código, con estandarización de nombres, plantillas, referencias cruzadas, evito ambigüedad.

7.5 Lecciones Aprendidas

El definir la arquitecta desde el principio economiza tiempo, recursos y es más eficiente que refactorizar todo después. Se aprendió también al documentar y tener bases, verificar antes de suponer. Varios errores e inconsistencias que se presentaron fueron la falta de estandarización, datos inventados o incoherencia en los nombres de campo. La idempotencia se tiene que diseñar desde el inicio si se trabajara con eventos, ya que la entrega puede repetirse. La seguridad granular se vuelve imprescindible, incluso si al inicio complica el arranque, pero es de suma importancia cuando conviven clientes, administradores y superadministradores. Además, definir un contrato de API explicito es la mejor forma de que clientes y servicios evolucionen juntos.

La arquitecta se refactorizó varias veces debido a la falta de planificación en la documentación. Así también se aprendió que las pruebas unitarias no reemplazan la validación de los contratos. Errores como que los refresh token no se guardaban, se tenían rutas desalineadas. En ocasiones se trabajaban con versiones diferentes del código causando conflicto en el trabajo compartido. El realizar los hallazgos con un ciclo de acciones correctivas, preventivas y de mejora de acuerdo en la norma ISO/IEC 29110 [ISO/IEC 2018], deja explícito que fallo, por qué y cómoc se halló la solución. Inclusive, una experiencia fue el aprender a fijar el valor de versión de java, ya que por esto una ejecución con JDK 27 falló al inicializar Loombok, ya que la base línea solo se produce con JDK 21.

7.6 Relación con Trabajos Previos

Los resultados concuerdan con lo que decía Newman [2021] y Fowler y Lewis [2014] respecto a la autonomía y cohesión funcional. La utilidad del API del Gateway y del consumidor idempotente que describe Richardson [2018]. La separación establecida entre dominio e infraestructura es una implementación explicita de la arquitectura Hexagonal [Cockburn 2005] y la inversión de dependencias [Martin 2017]. El panel de indicadores se apoya, con adaptaciones, en la cultura de medición de DevOps [Forsgren et al. 2018; Humble and Farley 2010]. Un pipeline de integración continua es un requisito indispensable para automatizar la recolección de las métricas DORA.

8. Conclusiones

8.1. Resumen de Hallazgo

Incluso en un contexto académico como en el que se construyó rentamovil, el aplicar DDD, arquitectura hexagonal y combinarla con microservicios, dio como resultado un sistema escalable, con alta mantenibilidad y, sobre todo, fácil de entender. El resultado de estas implementaciones dio como resultado una plataforma con cuatro servicios de negocio y un Gateway, 225 pruebas automatizadas realizadas sin fallos, además de un despliegue verificado y una integración manual documentada. El definir y separar el domino de la aplicación e infraestructura dejó clara y bien definidas las responsabilidades. Gracias al esquema por servicio se precisó quien era el dueño de los datos y así el Gateway se encargó de precisar lo transversal.

8.3 Limitaciones

Debido a que es un proyecto académico las limitaciones en recursos son bastante claras, además de no contar con una infraestructura de producción, por ende, la latencia no se pudo medir en condiciones reales. En las pruebas faltan las de contrato y las HTTP con MockMvc. El SQL Server funciona como una única instancia, sin alta disponibilidad (riesgo R-003). El limitador de solicitudes del Gateway trabaja en memoria y la URL del backend esta fija dentro del APK. Por última instancia, el volumen de usuarios es bajo, pero se espera explorar en contexto con alta concurrencia.

8.4 Trabajos Futuros

Se propone un orden, mejorar y aclarar las reglas de negocio actuales. Agregar pruebas MockMvc y de contrato, además de pruebas de persistencia para IAM, Booking y Billing, con Testcontainers [Testcontainers s. f.] para así tener compatibilidad real con SQL Server. Además de corregir errores en el contexto de telemetría debido a las pruebas faltantes dependientes del GPS. Cambiar el canal de eventos en memoria por broker con garantías de entrega. Implementar un pipeline de despliegue continuo en ambientes de producción. Así también, si el proyecto llegara a producción, habría que evaluar el paso gradual a una base de datos por servicio.

8.5 Reflexión Final

RentaMovil convierte un proceso manual en una plataforma organizada alrededor del dominio, con decisiones documentadas y límites establecidos. El aclarar, definir y establecer la arquitectura, reglas de negocio en la documentación permitió una mejor evolución en el desarrollo del sistema, así evitando ambigüedades, además de contradicciones debido al trabajo compartido, con la documentación como única fuente de verdad. Gracias a las métricas de calidad se puede explicar el sistema, probarlo y saber con exactitud que se podría mejorar o potenciar. La calidad de un producto no aparece sola, se sostiene con planeación, diseño y constancia.

Anexos A. Checklist de Reproducibilidad

Datos. Modelo relacional normalizado en Microsoft SQL Server base rentamovil, con datos de referencia sembrados por migraciones y datos sintéticos para pruebas.

Código. Repositorios independientes por servicio y por base de datos, arquitectura hexagonal documentada, convenciones de ramas y revisión por Pull Request.

Entorno. JDK 21, Maven 3.9.16, SQL Server en Docker, red Docker compartida rentamovil, Node.js para los clientes y Liquidbase para las migraciones.

Procedimiento. Levantar las bases y aplicar las migraciones en el orden documentado, arrancar los servicios y el Gateway, autentificarse con iam y recorrer los flujos críticos a través del Gateway. Para las pruebas automatizadas, ejecutar mvn -f <módulo>/pom.xml test en cada uno de los cinco módulos. Los reportes quedan en target/surefire-reports.

Resultados. Indicadores de calidad, contratos OpenAPI, historias de usuario y matrices de trazabilidad disponibles en el repositorio de documentación.

Referencias

AssertJ. s. f. AssertJ – Fluent assertions for Java. https://assertj.github.io/doc/

Alistair Cockburn. 2005. Hexagonal Architecture (Ports and Adapters). https://alistair.cockburn.us/hexagonal-architecture/

Mike Cohn. 2009. Succeeding with Agile: Software Development Using Scrum. Addison-Wesley.

R. D. C. Espinosa and J. C. C. Eraso. 2024. Gestión de tecnología y buenas prácticas en una empresa de desarrollo de software colombiana. Revista de los 17, 62 (2024).

Eric Evans. 2003. Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley.

Nicole Forsgren. 2021. DORA Metrics: The Four Keys to Software Delivery Performance. Google Cloud. https://dora.dev/

Nicole Forsgren, Jez Humble, and Gene Kim. 2018. Accelerate: The Science of Lean Software and DevOps. IT Revolution Press.

Martin Fowler. 2002. Patterns of Enterprise Application Architecture. Addison-Wesley.

Martin Fowler. 2012. TestPyramid. https://martinfowler.com/bliki/TestPyramid.html

Martin Fowler and James Lewis. 2014. Microservices. https://martinfowler.com/articles/microservices.html

Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. 1994. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

Jez Humble and David Farley. 2010. Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Addison-Wesley.

ISO/IEC. 2018. ISO/IEC 29110-4-1:2018, Systems and software engineering — Lifecycle profiles for Very Small Entities (VSEs) — Part 4-1:

Software engineering — Profile specifications: Generic profile group. International Organization for Standardization.

JaCoCo. s. f. JaCoCo Java Code Coverage Library. https://www.jacoco.org/jacoco/

Michael B. Jones, John Bradley, and Nat Sakimura. 2015. JSON Web Token (JWT). RFC 7519. Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc7519

JUnit. s. f. JUnit 5 User Guide. https://junit.org/junit5/docs/current/user-guide/

Liquibase. s. f. Liquibase Documentation. https://docs.liquibase.com/

Robert C. Martin. 2008. Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.

Robert C. Martin. 2017. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.

Gerard Meszaros. 2007. xUnit Test Patterns: Refactoring Test Code. Addison-Wesley.

Mockito. s. f. Mockito – Mocking framework for unit tests in Java. https://site.mockito.org/

Sam Newman. 2021. Building Microservices: Designing Fine-Grained Systems (2nd ed.). O'Reilly Media.

Michael Nygard. 2011. Documenting Architecture Decisions. Cognitect Blog. https://www.cognitect.com/blog/2011/11/15/documenting-architecture-decisions

OpenAPI Initiative. s. f. OpenAPI Specification. https://spec.openapis.org/oas/latest.html

OWASP Foundation. 2021. OWASP Top 10: The Ten Most Critical Web Application Security Risks. https://owasp.org/Top10/

David L. Parnas. 1972. On the Criteria To Be Used in Decomposing Systems into Modules. Communications of the ACM 15, 12 (1972), 1053–1058.

Chris Richardson. 2018. Microservices Patterns: With Examples in Java. Manning.

Ken Schwaber and Jeff Sutherland. 2020. The Scrum Guide. https://scrumguides.org/

Ian Sommerville. 2015. Software Engineering (10th ed.). Pearson.

Spring. s. f. Spring Boot Reference Documentation. https://docs.spring.io/spring-boot/

Testcontainers. s. f. Testcontainers – Integration tests with real dependencies. https://testcontainers.com/